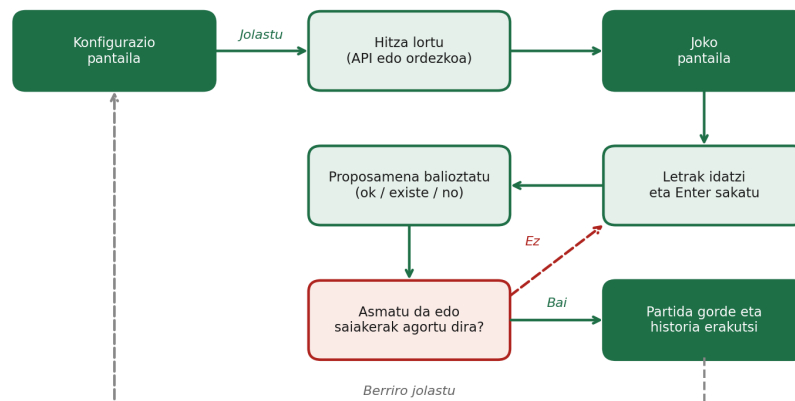


Dokumentazio teknikoa

Web aplikazioen garapena



Programaren funtzionamendu orokorraren diagrama

Wordle JS

Euskarazko hitzak asmatzeko jokia

HTML, CSS eta JavaScript bidezko web aplikazioa

Borja Suárez

Aurkibidea

1.	Sarrera	2
1.1.	Erabilitako teknologiak	2
1.2.	Fitxategien egitura	2
1.3.	Taldeko funtzioak	3
2.	Eskakizunen analisisa	3
2.1.	Eskakizun funtzionalak	3
2.2.	Eskakizun ez-funtzionalak	3
3.	Diseinua eta garapena	4
3.1.	Orriaren egitura (index.html)	4
3.2.	Estiloak (styles.css)	5
3.2.1.	Egoera-klaseak	5
3.2.2.	Hidden klase laguntzailea	5
3.3.	Jokoaren logika (code.js)	5
3.3.1.	Egoera-aldagaiak	5
3.3.2.	Gertaerak	6
3.3.3.	Hitz sekretua lortzea	6
3.3.4.	Taularen eta teklatuaren sorrera	7
3.3.5.	Sakatzeen kudeaketa	7
3.3.6.	Proposamenaren balioztapena	7
3.3.7.	Amaiera eta historiaren gordetzea	8
4.	Egindako probak	9
5.	Mugak eta hobekuntza-proposamenak	9
6.	Laburpena eta ondorioak	10
7.	Erreferentziak	11

1. Sarrera

Dokumentu honek **Wordle JS**-ren dokumentazio teknika jasotzen du. Hitzak asmatzeko joko ezagunaren web aplikazio bat da, eta hitz sekretua euskarazkoa izan dadin egokitu da. Jarraian, kontuan hartutako eskakizunak, programaren egitura, bere zati bakoitzaren funtzionamendua eta garapenean hartutako erabakiak azaltzen dira.

Programaren helburua da jokalaria saiakera kopurua eta hitzaren luzera konfiguratu ahal izatea, hitz sekretua proposamen jarraituen bidez asmatzea eta, amaitzean, azken partiden historia kontsultatzea. Dokumentazio hau taldeko edozein kidek, edo kanpoko enpresa batek, programaren egungo egoera ulertu eta zailtasunik gabe aldatu ahal izateko prestatu da.

1.1. Erabilitako teknologia

Nabigatzaileko teknologia estandarrak soilik erabiltzea erabaki dugu, liburutegi edo framework gehigarririk gabe, proiektuaren tamainak ez duelako hori justifikatzen eta horrela hedapena errazten delako (`index.html` fitxategia irekitzearekin nahikoa da).

Teknologia	Proiektuan duen erabilera
HTML5	Hiru pantailen egitura: konfigurazioa, jokia eta historia.
CSS3	Itxura bisuala, taula, pantailako teklatura eta egoera-koloreak.
JavaScript (ES6+)	Jokoaren logika, DOM manipulazioa, gertaerak eta dei asinkronoak.
Hitzen API REST-a	Luzera jakineko euskarazko hitz sekretua lortzea.
localStorage	Partiden historia nabigatzailean modu iraunkorrean gordetzea.

1. taula. Erabilitako teknologia.

1.2. Fitxategien egitura

Fitxategia	Edukia
index.html	Orriaren egitura eta kodeak erabiltzen dituen elementuak.
styles.css	Aplikazio osoaren estilo-orria.
code.js	Jokoaren logika osoa (egoera-aldagaiak, gertaerak eta funtzioak).
README.md	Iturburu-kodearen bertsio iruzkindua, lerroz lerro azalduta.

2. taula. Proiektua osatzen duten fitxategiak.

1.3. Taldeko funtzioak

[Hemen proiektuan parte hartu duten pertsonak eta bakoitzaren zeregina adierazi. Lana banakakoa izan bada, hala adierazi.]

2. Eskakizunen analisia

Programatu aurretik, aplikazioak eskaini behar zituen funtzionalitateak eta bete behar zituen baldintzak zehaztu ziren.

2.1. Eskakizun funtzionalak

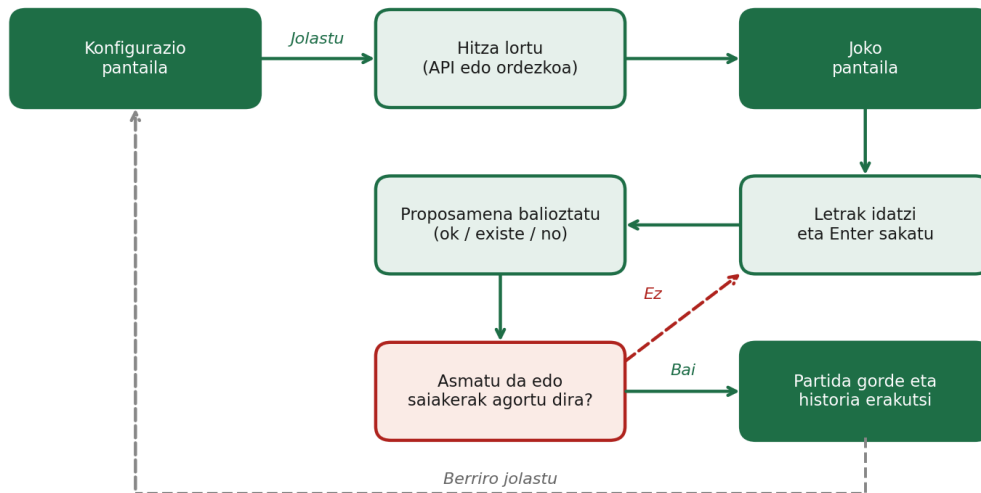
- **Partidaren konfigurazioa:** saiakera kopurua (1etik 10era) eta hitzaren luzera (3tik 8ra) hasi aurretik aukeratu ahal izan behar dira.
- **Hitz sekretua:** API batetik automatikoki lortu behar da eta euskarazkoa izan behar da.
- **Datu-sarrera:** letrak bai teklatu fisikoarekin bai pantailako teklatuarekin idatzi ahal izan behar dira; teklatuak Ñ eta azentudun bokalak izan behar ditu.
- **Balioztapena:** proposamen bakoitza koloreztatu behar da, letra bakoitza zuzena den (berdea), beste posizio batean dagoen (horia) edo ez dagoen (grisa) adieraziz.
- **Partidaren amaiera:** jokoa hitza asmatzean edo saiakerak agortzean amaitzen da, emaitzaren berri emanez.
- **Historia:** azken hamar partidak gorde behar dira, data, hitza, erabilitako saiakerak eta emaitza erakutsiz.
- **Partida berria:** orria birkargatu gabe beste partida bat hasi ahal izan behar da.

2.2. Eskakizun ez-funtzionalak

- Programak funtzionatzen jarraitu behar du APIak erantzuten ez badu ere; horregatik ordezeko hitz bat sartu da.
- Interfazeak tamaina desberdineko pantailetara egokitu behar du, mugikorak barne.
- Kodea antolatuta eta iruzkinduta egon behar da, mantentzeko erraza izan dadin.

3. Diseinua eta garapena

Atal honetan programaren funtzionamendua azaltzen da, fitxategien ordena jarraituz. 1. irudiak fluxu orokorra erakusten du: konfigurazio-pantailatik abiatzen da, hitz sekretua lortzen da, proposamenak idatzi eta balioztatzeko zikloa errepikatzen da eta, amaitzean, partida gorde eta historia erakusten da.



1. irudia. Partida baten fluxu orokorra.

3.1. Orriaren egitura (index.html)

Orria hiru blokeetan banatuta dago, eduki nagusi baten barruan. Une bakoitzean bat bakarrik dago ikusgai: gainerakoak **hidden** CSS klasearekin ezkutatuta egoten dira, eta JavaScript kodeak partidaren fasearen arabera gehitu edo kentzen du.

Elementua (id)	Mota	Funtzioa
konfig-inprimakia	form	Hasierako inprimakia, saiakeren (saiakerak) eta luzeraren (luzera) eremuekin.
joko-eremua	div	Jokoaren edukiontzia; taula eta teklatura ditu. Ezkutatuta hasten da.
taula	div	Errenkadak eta laukiak dinamikoki sortzen diren edukiontzia.
teklatura	div	Pantailako teklatura sortzen den edukiontzia.
historia-eremua	div	Historiaren amaierako pantaila. Ezkutatuta hasten da.
historia-zerrenda	ul	Aurreko partidak txertatzen diren zerrenda.
btn-berrir-jolastu	button	Inprimakira itzuli eta beste partida bat hasteko botoia.

3. taula. HTMLaren elementu nagusiak.

Informazioa

Inprimakiko eremu numerikoek **min**, **max** eta **required** atributuak dituzte; horrela, nabigatzaileak berak eragozten du tartetik kanpoko balioak (1etik 10era saiakerak eta 3tik 8ra letrak) bidaltzea.

3.2. Estiloak (styles.css)

Estilo-orriak itxura sinple eta irakurterraza definitzen du. **Flexbox** erabili da edukia zentratzeko eta taulako eta teklatuko errenkadak antolatzeko, eskuragarri dagoen zabalerara moldatzea ahalbidetzen baitu eskuzko kalkulurik gabe.

3.2.1. Egoera-klaseak

Laukien eta teklen kolorea hiru klaserekin bakarrik kontrolatzen da; JavaScript kodeak klase horiek gehitzen ditu proposamen bakoitza balioztatu ondoren. Aukeratutako koloreak jatorrizko jokoarenak dira, ezagunak egin daitezten.

Klasea	Kolorea	Esanahia
ok	Berdea (#6aaa64)	Letra zuzena da eta posizio zuzenean dago.
existe	Horia (#c9b458)	Letra hitzean dago, baina beste posizio batean.
no	Grisa (#787c7e)	Letra ez dago hitzean.

4. taula. Egoera-klaseak CSSn.

3.2.2. Hidden klase laguntzailea

hidden klaseak **display: none** **!important** aplikatzen du. **!important** adierazpena justifikatuta erabili da: ezkutatutako elementu bat ez dadin erakutsi, nahiz eta espezifikagoa den beste arau batek (adibidez, **card** klasearenak) bere bistaratzea aldatzen saiatu.

3.3. Jokoaren logika (code.js)

Fitxategia bost blokeetan antolatzen da: egoera-aldagaiak, DOM elementuak, gertaerak, jokoaren funtzioak eta balioztapen- eta amaiera-funtzioak.

3.3.1. Egoera-aldagaiak

Aldagai globalek jokatzen ari den partidaren informazioa gordetzen dute. Funtzioetatik bereizi dira, kodearen edozein zatik kontsultatu eta eguneratu ahal ditzan.

Aldagaia	Hasierako balioa	Azalpena
helburuHitza	""	Partidaren hitz sekretua, maiuskulaz.
gehienezkoSaiakerak	6	Jokalariak aukeratutako gehieneko saiakera kopurua.
hitzLuzera	5	Hitz sekretuaren luzera.
unekoSaiakera	0	Uneko errenkadaren indizea (0 lehen errenkada da).
unekoProposamena	""	Uneko errenkadan orain arte idatzitako letrak.

5. taula. Egoera-aldagaiak.

3.3.2. Gertaerak

Aplikazioak hiru gertaera motari erantzuten die:

- **Inprimakiaren submit gertaera:** balioak irakurtzen ditu, hitz sekretua eskatzen du, pantaila aldatzen du eta taula eta teklatua eraikitzen ditu. **preventDefault** deitzen zaio orria birkargatu ez dadin.
- **Dokumentuaren keydown gertaera:** teklatu fisikoa jasotzen du. Joko-pantaila ezkutatuta badago, ez ikusiarena egiten da. Baliozko letrak adierazpen erregular batekin egiaztatzen dira, Ñ eta azentudun bokalak onartzen dituenak.
- **«Berriro jolastu» botoiaren click gertaera:** kontagailuak berrezartzen ditu eta inprimakira itzultzen da.

Bai teklatu fisikoak bai pantailako teklatuak beren sakatzeak **prozesatuTeklaPultsaketa** funtzio berera bidaltzen dituzte. Erabaki horrek kodea bikoiztea saihesten du eta bi sarrera-moduek portaera berbera izatea ziurtatzen du.

3.3.3. Hitz sekretua lortzea

lortuHitza funtzio asinkronoak **fetch** eskaera egiten dio APIari, hizkuntza (**eu**), luzera eta nahi den hitz kopurua adieraziz:

```
https://words-api-sy2x.onrender.com/api/word?lang=eu&length=5&number=1
```

1. kodea. Hitzen APIari egindako eskaera (adibidea, 5 luzerarekin).

Erantzuna zerrenda ala objektu bat izango den ziur ez dakigunez, kodeak bi formatuak kontuan hartzen ditu (**"HITZA"**) eta **{ word: "HITZA" }**) eta emaitza maiuskulara bihurtzen du. Eskaerak huts egiten badu, errorea harrapatzen da eta ordeko hitz bat itzultzen da, jokia eten ez dadin.

3.3.4. Taularen eta teklatuaren sorrera

hasieratuTaula funtzioak taula bi begizta habiaratuen bidez sortzen du: kanpokoak errenkada bat sortzen du saiakera bakoitzeko eta barnekoak laukia bat letra bakoitzeko. Laukiei identifikatzaile bakarra esleitzen zaie, **laukia-errenkada-zutabea** formatuarekin; adibidez, **laukia-0-3** lehen saiakeraren laugarren letra da. Formatu horri esker, edozein laukia zuzenean aurki daiteke DOMa zeharkatu gabe.

hasieratuTeklatua funtzioak teklen banaketa duen matrize bat zeharkatzen du eta botoi bat sortzen du tekla bakoitzeko. Banaketak azentudun bokalak eta Ñ letra dituen errenkada gehigarri bat barne hartzen du. Tekla bakoitzaren balioa **data-key** atributuan gordetzen da, gero aurkitu eta koloreztatu ahal izateko.

3.3.5. Sakatzeen kudeaketa

prozesatuTeklaPultsaketa funtzioak honako arauak aplikatzen ditu:

Tekla	Baldintza	Ekintza
Enter	Proposamenak hitzLuzera letra ditu zehazki.	balioztatuProposamena funtzioari deitzen zaio.
Del	Proposamena ez dago hutsik.	Azken letra ezabatzen da eta errenkada eguneratzen da.
Letra	Proposamenak hitzLuzera baino letra gutxiago ditu.	Letra gehitzen da eta errenkada eguneratzen da.

6. taula. Sakatzeen kudeaketa-arauak.

3.3.6. Proposamenaren balioztapena

balioztatuProposamena funtzioa programaren zatirik delikatuena da, errepikatutako letrak behar bezala tratatu behar dituelako. Horretarako lau pasaldi egiten dira, jarraian azaltzen direnak.

- **Lehen pasaldia (letra zuzenak):** proposamenaren letra bakoitza hitz sekretuko posizio bereko letrarekin alderatzen da. Bat badatoz, **ok** gisa markatzen da eta hitz sekretuaren kopian letra hori ezabatzen da (**null** balioarekin ordezkatzen da), berriz erabili ahal izan ez dadin.
- **Bigarren pasaldia (badauden letrak):** markatu gabeko letretarako, letra hori hitz sekretuan geratzen denean bilatzen da. Aurkitzen bada, **existe** gisa markatzen da eta agerraldi hori ezabatzen da.
- **Hirugarren pasaldia (koloreak):** dagokion klasea gehitzen zaio laukiari eta pantailako teklutako tekla eguneratzen da, lehentasun bat errespetatuz: berdea ez da inoiz gainidazten eta horia berdeak bakarrik ordezka dezake.

- **Laugarren pasaldia (amaiera edo aurrerapena):** proposamena hitzarekin bat badator, partida irabazi egiten da; bestela, hurrengo errenkadara pasatzen da eta, saiakerarik geratzen ez bada, galdu egiten da.

Adi!

Lehen bi pasaldien ordena ezinbestekoa da. Letra horiak berdeak baino lehen markatuko balira, errepikatutako letra batek posizio zuzen bati dagokion agerraldia kontsumitu lezake eta emaitza okerra izango litzateke.

1. adibidea. Hitz sekretua **LIBRO** da eta proposamena **RIBOL**. Lehen pasaldian I eta B zuzen gisa markatzen dira. Bigarrenean, R, O eta L beste posizio batzuetan daude. Eraitza: horia, berdea, berdea, horia, horia.

2. adibidea (letra errepikatuak). Hitz sekretua **PLAZA** da eta proposamena **AAAAA**. Lehen pasaldian 3. posizioako A bakarrik markatzen da zuzen gisa. Bigarrenean, erabili gabeko A bakar bat geratzen da, eta proposamenaren lehen A-ri esleitzen zaio. Eraitza: horia, grisa, berdea, grisa, grisa. Adibide honek frogatzen du programak ez duela letra baten hitzean benetan daudenak baino agerraldi gehiago markatzen.

3.3.7. Amaiera eta historiaren gordetzea

Partida amaitzean, **amaituJokoa** funtzioak 200 milisegundo itxaroten ditu azken errenkadaren koloreak ikusi ahal izateko, emaitzaren mezu bat erakusten du, partida gordetzen du eta historiaren pantaila aurkezten du.

Historia **localStorage**-n gordetzen da, **wordle_historia** gakoarekin, JSON testu bihurtutako objektu-zerrenda gisa, biltegiratze horrek kateak bakarrik onartzen baititu. Erregistro bakoitzak egitura hau du:

```
{
  hitza: "LIBRO",
  saiakerak: 4,
  data: "05/10/2026",
  emaitza: "irabazi" // edo "galdu"
}
```

2. kodea. Historiako erregistro baten egitura.

Erregistro berriak zerrendaren hasieran txertatzen dira eta, hamar gainditzen direnean, zaharrena ezabatzen da. Historiaren tamaina mugatu da amaierako pantaila irakurgarria izan dadin eta erabiltzailearen nabigatzailean daturik mugarik gabe ez pilatzeko.

Jokalariak irabazten duenean, saiakera-kontagailua ez da oraindik handitu, beraz bat gehitzen da erregistroa gordetzean. Galtzen duenean, kontagailuak gehieneko balioa lortuta dauka eta ez da aldatzen.

4. Egindako probak

Programak eskakizunak betetzen dituela egiaztatzeko, proba-kasu hauek definitu ziren. Kodea aldatzen den bakoitzean errepikatzea gomendatzen da.

Zk.	Proba-kasua	Espero den emaitza
1	Inprimakia lehenetsitako balioekin bidaltzea (6 eta 5).	5 laukiko 6 errenkadako taula eta teklatua agertzen dira.
2	12 saiakera edo 2 letra sartzea.	Nabigatzaileak bidalketa baztertzen du, tartetik kanpo dagoelako.
3	Enter sakatzea proposamena osatu gabe dagoenean.	Ez da ezer gertatzen; proposamena ez da balioztatzen.
4	Idaztea eta Del sakatzea.	Azken letra laukitik desagertzen da.
5	Letra errepikatuak dituen hitz bat proposatzea (2. adibidea).	Koloreak 3.3.6 ataleko berberak dira.
6	Hitza asmatzea.	Garaipenaren mezua; historiak «irabazi» erakusten du.
7	Saiakera guztiak agortzea.	Porrotaren mezua, hitzarekin; historiak «galdu» erakusten du.
8	Hamaika partida jokatzeko.	Historiak azken hamar partidak bakarrik gordetzen ditu.
9	Sarea deskonektatzea eta partida hastera.	Ordezko hitza erabiltzen da eta jokoak aurrera jarraitzen du.
10	«Berrir jolastu» sakatzea.	Inprimakia agertzen da eta partida berria teklatu garbiarekin hasten da.

7. taula. Proba-kasuak.

5. Mugak eta hobekuntza-proposamenak

Kodea berrikusi ondoren, hobetu daitezkeen alderdi batzuk antzeman dira; jarraian azaltzen dira, proposatzen dugun konponbidearekin batera. Horietako batek ere ez dio eragozten jokoaren ohiko funtzionamenduari.

Antzemandako alderdia	Azalpena	Hobekuntza proposatua
Ordezko hitza	Funtzioak beti zerrendako lehen hitza hautatzen du, luzeraren arabera moztu edo A letrekin betetzen du (adibidez, «CASAAAA» 8 letrarekin) eta gaztelaniazkoa da, ez euskarazkoa.	Luzera bakoitzeko euskarazko hitzen zerrenda sortzea eta ausaz bat aukeratzea.
Hiztegi-balioztapenik eza	Edozein letra-konbinazio onartzen da proposamen gisa, hitz erreala ez bada ere.	Proposamena baliozko hitzen zerrenda batekin egiaztatzea.
Hitza kontsolan ikusgai	Hitz sekretua console.log bidez erakusten da, probak errazteko.	Lerro hori amaierako bertsioan kentzea.
alert bidezko mezuak	Leihoak orria blokeatzen du eta aplikazioaren estetika hausten du.	Interfazean txertatutako mezu batekin ordeztea.
Pantailako teklatuaren fokua	Pantailako tekla bat sakatu ondoren, botoiak fokua mantentzen du; ondoren teklatu fisikoan Enter sakatzen bada, nabigatzaileak botoi hori berriro aktibatu dezake.	Klik bakoitzaren ondoren botoiaren fokua kentzea (blur) edo preventDefault bidez eragozteak.
Asmatu ondorengo sarrera	Azken 200 ms-ko itxaronaldian sakatzeak onartzen dira oraindik.	Partidaren amaiera hasten denean sarrera blokeatzea.
APIaren atzerapena	Zerbitzua inaktibo badago, lehen erantzunak denbora har dezake eta itxaronaldiaren berri ez da ematen.	Hitza eskatzen den bitartean karga-adierazle bat erakustea.

8. taula. Antzemandako mugak eta hobekuntza-proposamenak.

Kontuz!

Ordezko hitzaren hobekuntza da gomendagarriena, gaur egun APIak huts egiten badu hitz errepikatua eta euskaraz baliozkoa ez den hitz bat duen partida bat sortzen baita.

6. Laburpena eta ondorioak

Hitzak asmatzeko joko oso bat garatu da, planteatutako eskakizun guztiak betetzen dituen: partidaren konfigurazioa, hitzaren lorpena API batetik, teklatu fisikoko eta pantailako sarrera, koloreekin balioztatzea, partidaren amaieraren kudeaketa eta historiaren iraunkortasuna.

Erabaki egokienak izan direla uste dugu: sakatzeen kudeaketa funtzio bakar batean zentralizatzea, laukia bakoitza identifikatzaile aurreikus daitekeen batekin identifikatzea eta balioztapena bi pasaldietan ebaztea, horrek letra errepikatuen tratamendu zuzena bermatzen baitu. Gainera, teknologia estandarrak soilik erabiltzeak programa arina eta edozein nabigatzailetan exekutatzeko erraza egiten du.

Etorkizuneko lan gisa, aurreko atalean jasotako hobekuntzei heltzea gomendatzen da, bereziki ordezkotzen zerrendari eta proposamenen balioztapenari, jokoaren sendagoa izan dadin eta jatorrizko jokoa esperientziara gehiago hurbil dadin.

7. Erreferentziak

- Erabilgarritako hitzen APIa: <https://words-api-sy2x.onrender.com>
- MDN Web Docs. JavaScript, DOM, fetch eta localStorage dokumentazioa: <https://developer.mozilla.org>
- Gómez Olivencia, R. (2025). Nola egin dokumentazio ona. Lanbide Heziketa: Informatika eta Komunikazioak. Dokumentu honen egitura eta formatua zehazteko jarraitutako gida.